Esercitazione di Genetic Programming

Corso di Intelligenza Artificiale

Federico Sassi, PhD

2016-12-1

Outline

1. Genetic Programming
2. Esercitazione 3
3. Proposta progetti AI

Outline

1. Genetic Programming
2. Esercitazione 3
3. Proposta progetti AI

Genetic Programming - recall

1. Generate an initial population of random compositions of the functions and terminals of the problem (computer programs).
2. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
3. Create a new population of computer programs.
   1. Copy the best existing programs (optional: elistism)
   2. Create new computer programs by mutation
   3. Create new computer programs by crossover(sexual reproduction).
4. The best computer program that appeared in any generation, the best-so-far solution, is designated as the result of genetic programming [Koza 1992].

Genetic Programming - recall

1. Generate an initial population of random compositions of the functions and terminals of the problem (computer programs).
2. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
3. Create a new population of computer programs.
   1. Copy the best existing programs (optional: elistism)
   2. Create new computer programs by mutation
   3. Create new computer programs by crossover(sexual reproduction).
4. The best computer program that appeared in any generation, the best-so-far solution, is designated as the result of genetic programming [Koza 1992].

Genetic Programming - recall

1. Generate an initial population of random compositions of the functions and terminals of the problem (computer programs).
2. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
3. Create a new population of computer programs.
   1. Copy the best existing programs (optional: elistism)
   2. Create new computer programs by mutation
   3. Create new computer programs by crossover(sexual reproduction).
4. The best computer program that appeared in any generation, the best-so-far solution, is designated as the result of genetic programming [Koza 1992].

Genetic Programming - recall

1. Generate an initial population of random compositions of the functions and terminals of the problem (computer programs).
2. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
3. Create a new population of computer programs.
   1. Copy the best existing programs (optional: elistism)
   2. Create new computer programs by mutation
   3. Create new computer programs by crossover(sexual reproduction).
4. The best computer program that appeared in any generation, the best-so-far solution, is designated as the result of genetic programming [Koza 1992].

Genetic Programming - recall

1. Generate an initial population of random compositions of the functions and terminals of the problem (computer programs).
2. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
3. Create a new population of computer programs.
   1. Copy the best existing programs (optional: elistism)
   2. Create new computer programs by mutation
   3. Create new computer programs by crossover(sexual reproduction).
4. The best computer program that appeared in any generation, the best-so-far solution, is designated as the result of genetic programming [Koza 1992].

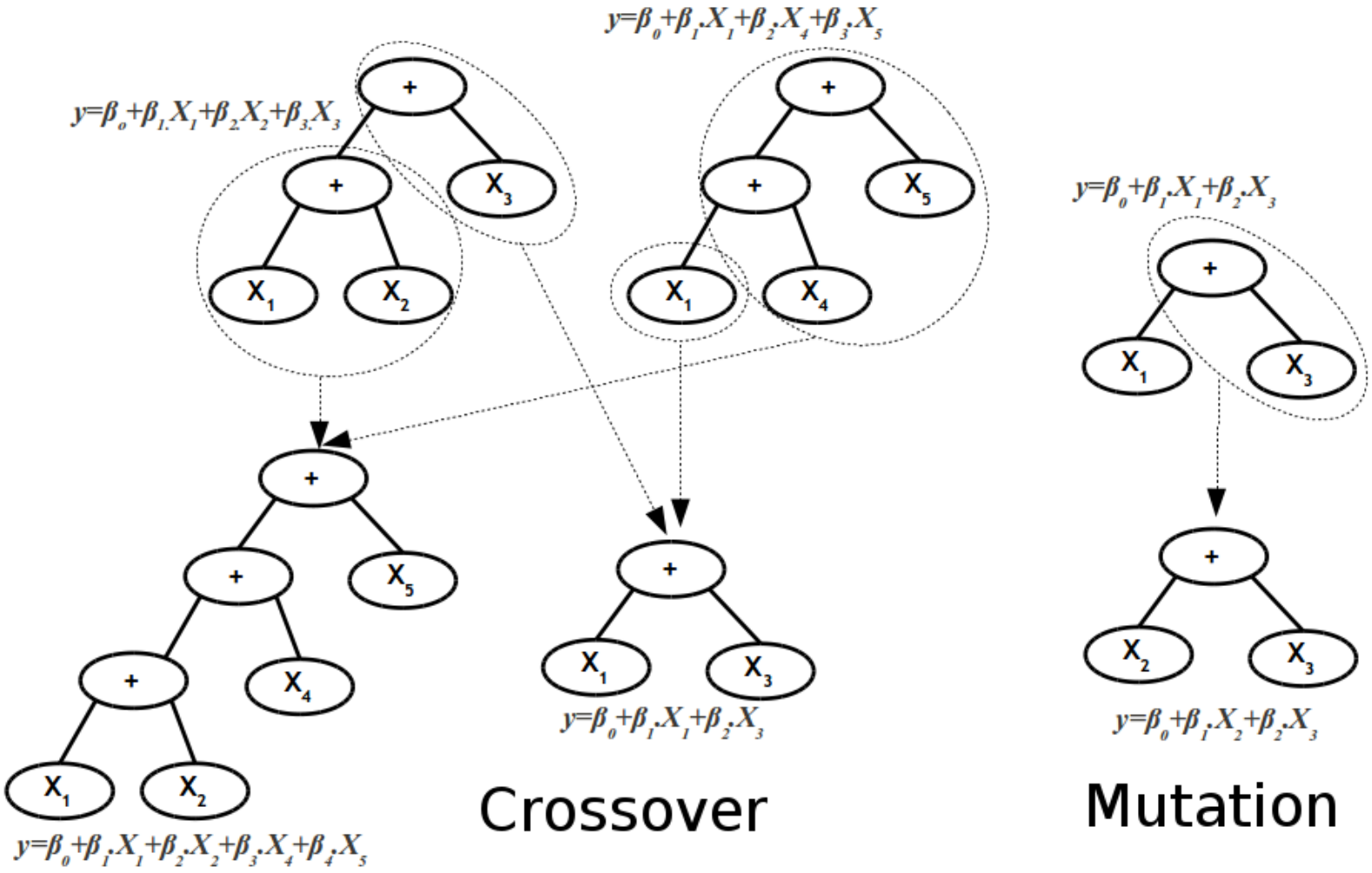
Genetic Programming - recall

1. Generate an initial population of random compositions of the functions and terminals of the problem (computer programs).
2. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
3. Create a new population of computer programs.
   1. Copy the best existing programs (optional: elistism)
   2. Create new computer programs by mutation
   3. Create new computer programs by crossover(sexual reproduction).
4. The best computer program that appeared in any generation, the best-so-far solution, is designated as the result of genetic programming [Koza 1992].

Genetic Programming - recall

1. Generate an initial population of random compositions of the functions and terminals of the problem (computer programs).
2. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
3. Create a new population of computer programs.
   1. Copy the best existing programs (optional: elistism)
   2. Create new computer programs by mutation
   3. Create new computer programs by crossover(sexual reproduction).
4. The best computer program that appeared in any generation, the best-so-far solution, is designated as the result of genetic programming [Koza 1992].

Genetic Programming - Operators



DEAP

deap
Distributed Evolutionary Algorithms in Python

- Genetic algorithm
- Genetic programming using prefix trees
- Evolution strategies
- Multi-objective optimisation
- Co-evolution (cooperative and competitive) of multiple populations
- Parallelization of the evaluations (and more)
- etc...

- https://code.google.com/p/deap/
- https://github.com/DEAP/deap

Outline

1. Genetic Programming
2. Esercitazione 3
3. Proposta progetti AI

Esercitazione 3: FILTRAGGIO DI IMMAGINI

Data l'immagine a livelli di grigio $I(x, y)$ e una sua versione rumorosa $O(x, y) = I(x, y) + N(15)$, con $N(15)$ rumore uniforme compreso in $[-15, 15]$ (limitando comunque i valori di $O(x, y)$ all'intervallo $[0, 255)$), trovare una funzione $f(x, y)$ tale che

$I(x, y) = f(O(x, y))$ sia il piu' possibile simile a $I(x, y)$.

T = O(x-1,y-1), O(x-1,y), O(x-1,y+1), O(x,y-1), O(x,y), O(x,y+1), O(x+1,y-1), O(x+1,y), O(x+1,y+1), $K_i i = 1, n$ con K_i costanti, n arbitrario.

Limitare il codominio delle funzioni incluse in f all'intervallo $[0, 255)$ e la profondita' degli alberi generati a 5.

Esercitazione 3: FILTRAGGIO DI IMMAGINI

original - noisy

- dataset creation script is included in the tar.gz updated!
- dataset is a ”matrix of numbers”
- can be seen as a regression problem!

Esercitazione 3: FILTRAGGIO DI IMMAGINI

original - noisy

- dataset creation script is included in the tar.gz updated!
- dataset is a ”matrix of numbers”
- can be seen as a regression problem!

Esercitazione 3: FILTRAGGIO DI IMMAGINI

original - noisy

- dataset creation script is included in the tar.gz updated!
- dataset is a ”matrix of numbers”
- can be seen as a regression problem!

DEAP tutorial

ToDo list

- create dataset
- create function nodes and terminal nodes
- fitness function
- setup DEAP for desired type of population, individuals, generation policies
- setup selection operator, crossover and mutation
- start evolution!

Create dataset: create_dataset.py

```
#LOADIMAGE USING PIL AND CONVERT TO NUMPY ARRAY

NOISE_LEVEL = 15
noise = numpy.random.randint(-NOISE_LEVEL,NOISE_LEVEL+1,img.size)

bnarray_noise = bnarray+noise

#fixing limits
print "fixing limits..."
if bnarray_noise.min() < 0 or bnarray_noise.max() > 255:
    print "\t********* reshaping preserving noise dynamic"
    bnarray_noise= 255*(bnarray_noise-bnarray_noise.min())/
                   (bnarray_noise.max()-bnarray_noise.min())

#SAVE NUMPY ARRAY USING SAVETXT AND CONVERT TO IMAGE
```

function nodes: + - * / avg2 avg3

examples:

```
def safeadd(a, b):
    s = a + b
    if s > 255:
        s = 255
    return s

def safediv(a, b):
    try:
        s = a / b
    except ZeroDivisionError:
        return 0
    if s < 0:
        return 0
    if s > 255:
        return 255
```

Create function nodes: + - * / avg2 avg3

```
#see later
pset = gp.PrimitiveSet("MAIN", 9)
#....
pset.addPrimitive(safeadd, 2)
pset.addPrimitive(safesub, 2)
pset.addPrimitive(safemul, 2)
pset.addPrimitive(safediv, 2)
pset.addPrimitive(avg2, 2)
pset.addPrimitive(avg3, 3)
```

you need to implement your custom function or use (if possible) standard ones like $pset.addPrimitive(operator.sub, 2)$

Create terminal nodes

```
pset = gp.PrimitiveSet("MAIN", 9)
#defines all variables = 9 pixel!

pset.renameArguments(ARG0='x0y0') #O(x-1,y-1)
pset.renameArguments(ARG1='x0y')  #O(x-1,y)
pset.renameArguments(ARG2='x0y1') #O(x-1,y+1)
pset.renameArguments(ARG3='xy0')  #O(x,y-1)
pset.renameArguments(ARG4='xy')   #O(x,y)
pset.renameArguments(ARG5='xy1')  #O(x,y+1)
pset.renameArguments(ARG6='x1y0') #O(x+1,y-1)
pset.renameArguments(ARG7='x1y')  #O(x+1,y)
pset.renameArguments(ARG8='x1y1') #O(x+1,y+1)

# An ephemeral constant is a terminal
# encapsulating a value that is generated
# from a given function at run time
pset.addEphemeralConstant("rand130",
                  lambda: random.randint(1,30))
```

Fitness function

```
#individual: GP tree
#points: all pixel coordinates to be evaluateds
def evalImage(individual, points):
 func = toolbox.compile(expr=individual)
 sqerrors = 0
 for p in points:
   try:
     #distance from the CLEAN image
     sqerrors+=numpy.power((func(datanoise[p[0]-1,p[1]-1],
                                  datanoise[p[0]-1,p[1]],
                                  ....
                                 datanoise[p],
                                 ....
                                 datanoise[p[0]+1,p[1]+1] )
                                 - dataclean[p] ), 2 )
   except:
     sqerrors += 1000.0 #numpy.inf

 return sqerrors / len(points),
```

DEAP setup 1/3

```
#minimizing fitness function
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
#population individuals are TREEs
creator.create("Individual", gp.PrimitiveTree,
                                fitness=creator.FitnessMin)

toolbox = base.Toolbox()
#generation method: HalfAndHalf max 3 depth
toolbox.register("expr", gp.genHalfAndHalf, pset=pset,
                                            min_=1, max_=3)
toolbox.register("individual", tools.initIterate,
                         creator.Individual, toolbox.expr)
toolbox.register("population", tools.initRepeat,
                               list, toolbox.individual)
#when "compiling" individuals we assign the terminal set
toolbox.register("compile", gp.compile, pset=pset)
```

DEAP setup 2/3

```
#generate the list of pixel coordinates on which
#evaluate the fitness function
toolbox.register("evaluate", evalImage, points=
  [(y,x) for y in range(1,SIZE-1) for x in range(1,SIZE-1)])

#parameters of selection operator: TOURNAMENT
toolbox.register("select", tools.selTournament, tournsize=5)
# CROSSOVER type
toolbox.register("mate", gp.cxOnePoint)
# MUTATION size constrain and parameters
toolbox.register("expr_mut", gp.genFull, min_=1, max_=3)
toolbox.register("mutate", gp.mutUniform, expr=toolbox.expr_mut,
                                                  pset=pset)

# limit on tree depth from exercise 3
toolbox.decorate("mate", gp.staticLimit(
                key=operator.attrgetter("height"), max_value=5))
toolbox.decorate("mutate", gp.staticLimit(
                key=operator.attrgetter("height"), max_value=5))
```

DEAP setup 3/3

```
# ENABLE PARALLEL PROCESSING
pool = multiprocessing.Pool()
toolbox.register("map", pool.map)

#this is enough to exploit all cores on your pc
```

DEAP evolution 1/4

```
def main():
    # Start a new evolution
    population = toolbox.population(n=POPSIZE)
    start_gen = 0
    #to save best individual
    halloffame = tools.HallOfFame(maxsize=1)
    logbook = tools.Logbook()
    #to collect statistics
    stats_fit = tools.Statistics(lambda ind: ind.fitness.values)
    stats_size = tools.Statistics(len)
    mstats = tools.MultiStatistics(fitness=stats_fit, size=stats_s
    mstats.register("avg", numpy.mean)
    mstats.register("std", numpy.std)
    mstats.register("min", numpy.min)
    mstats.register("max", numpy.max)

    #continue...
```

DEAP evolution 2/4

Part of an evolutionary algorithm applying only the variation part (**crossover and mutation**). The modified individuals have their fitness invalidated. The individuals are cloned so returned population is independent of the input population.

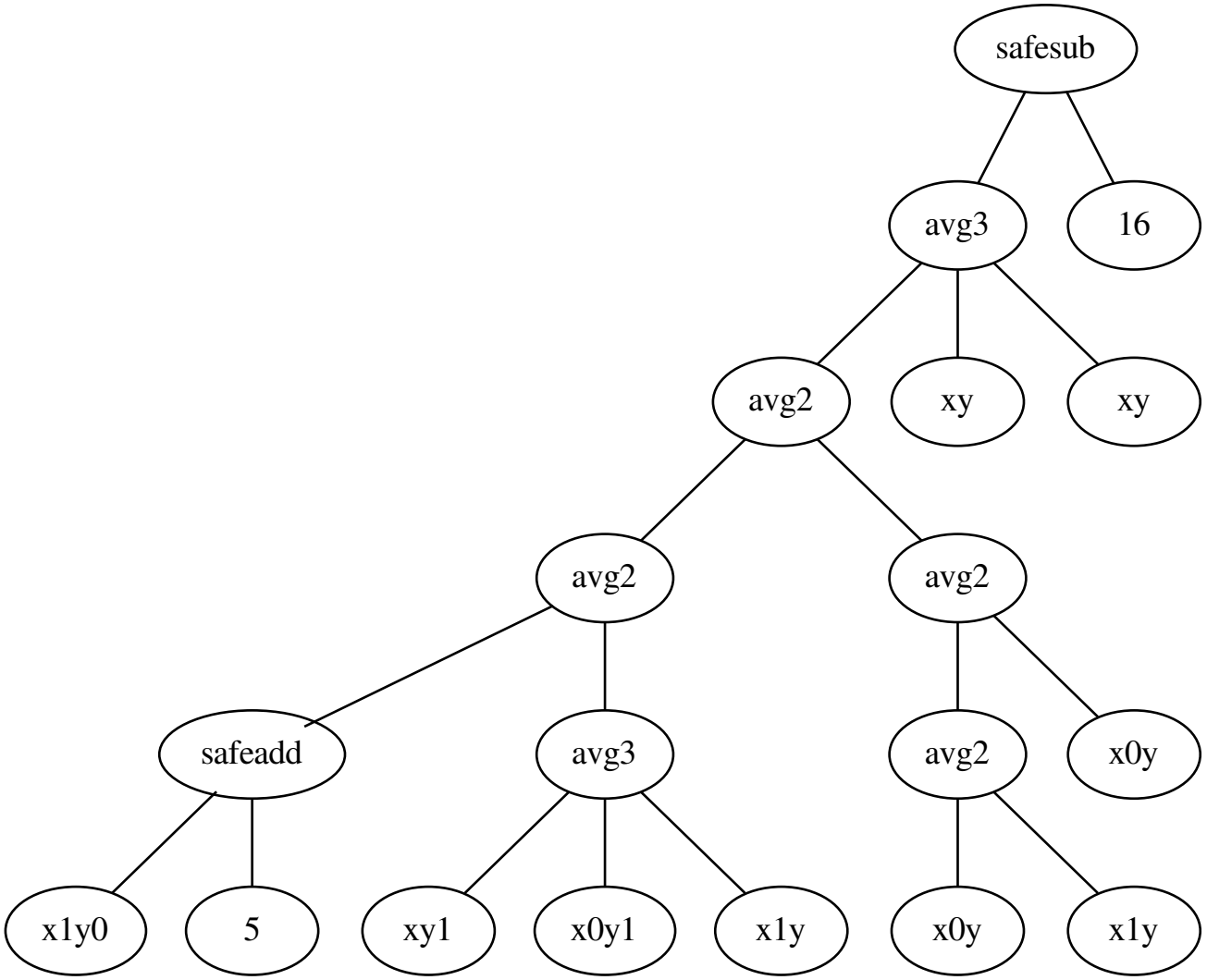
```
#for each generation up to defined in NGEN by user
for gen in range(start_gen, NGEN):
    population = algorithms.varAnd(population, toolbox,
        cxpb=CXPB, mutpb=MUTPB)
    #continue...
```

The variation goes as follow. First, the parental population P_{p} is duplicated using the *toolbox.clone()* method and the result is put into the offspring population P_{o}. A first loop over P_{o} is executed to mate consecutive individuals. According to the crossover probability *cxpb*, the individuals $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ are mated using the *toolbox.mate()* method. The resulting children $\mathbf{y}_i$ and $\mathbf{y}_{i+1}$ replace their respective parents in P_{o}. A second loop over the resulting P_{o} is executed to mutate every individual with a probability *mutpb*. When an individual is mutated it replaces its not mutated version in P_{o}. The resulting P_{o} is returned.

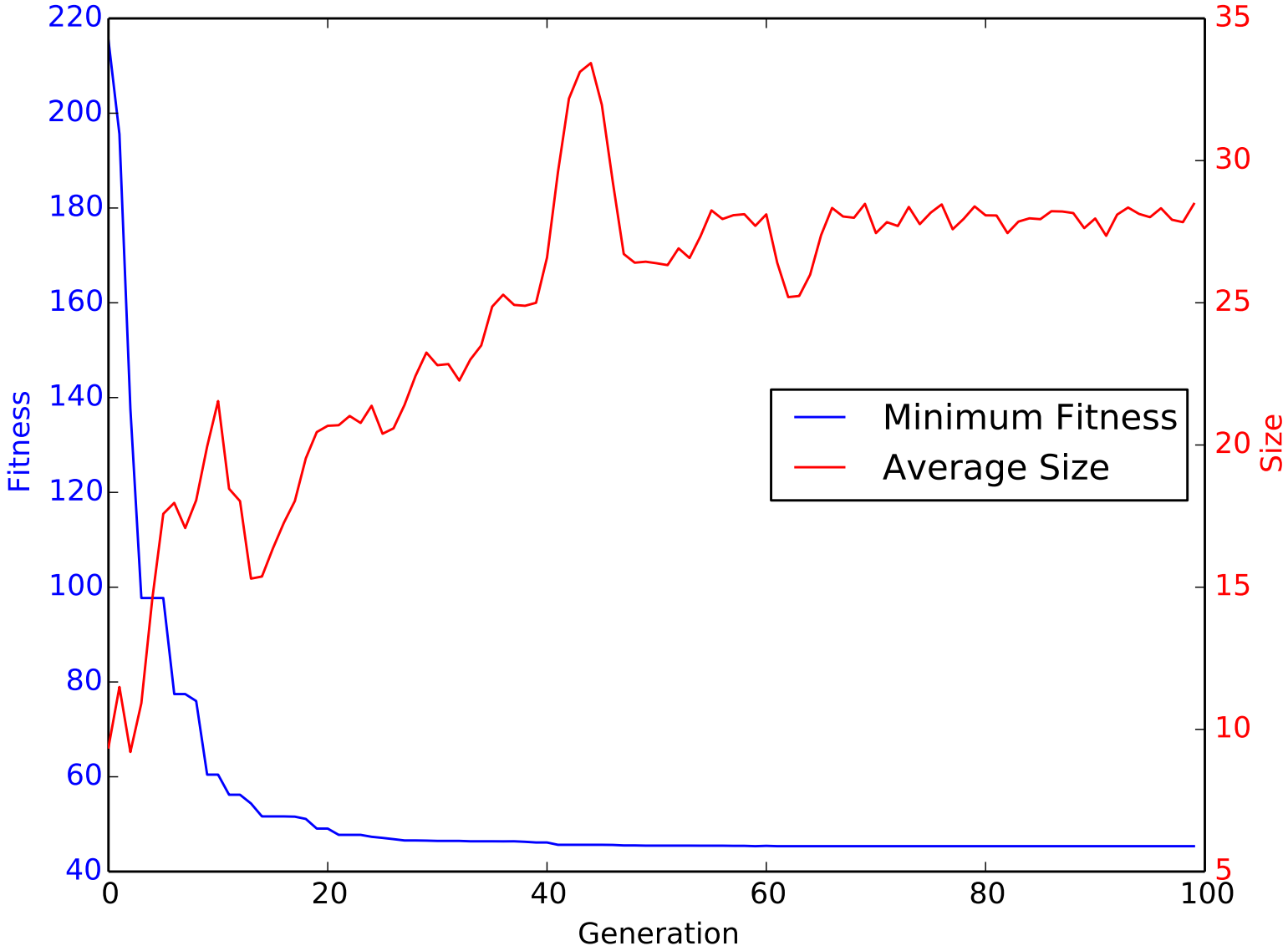
DEAP evolution 3/4

```
for gen in range(start_gen, NGEN):
    population = algorithms.varAnd(population, toolbox,
        cxpb=CXPB, mutpb=MUTPB)
    # Evaluate the individuals with an invalid fitness
    invalid_ind = [ind for ind in population
                                    if not ind.fitness.valid]
    #parallel computation of fitness if enables
    fitnesses = toolbox.map(toolbox.evaluate, invalid_ind)

    #copy back calculated values
    for ind, fit in zip(invalid_ind, fitnesses):
        ind.fitness.values = fit

    #keep track of the best and collect statistics
    halloffame.update(population)
    record = mstats.compile(population)
    logbook.record(gen=gen, evals=len(invalid_ind), **record)

    #Select k individuals from the input individuals using
    # k tournaments of tournsize individuals.
    population = toolbox.select(population, k=len(population))
```

DEAP results



DEAP results

original - noisy - post-GP

DEAP results